

Chemnitzer Linux-Tage 2026

Von der Vogel-Kamera zum eigenen KI-Modell

Souveräne Edge-AI
auf Open-Source-Basis mit Raspberry Pi

Adrian Imme & Roland Imme

29. März 2026 ■ Raum V7



Was macht jeder mit KI?

- ▶ ChatGPT & Large Language Models
- ▶ Generative AI für Bilder (DALL-E, Midjourney)
- ▶ AI-Coding-Assistenten (GitHub Copilot)
- ▶ Machine Learning as a Service (MLaaS)
- ▶ Autonome Fahrzeuge & Robotik
- ▶ AI-Voice-Assistenten & Sprachsynthese

Und wir?

Praktische Anwendung mit Raspberry Pi!

- ▶ Ziel des Vortrags (Erfahrungsaustausch)
- ▶ Motivation: Warum Vogel-Videos?
- ▶ Entstehung durch anderes Projekt:
 - ▶ Kamerawagen-Linux
 - ▶ Erfahrungen mit Raspberry Pi Kameras
 - ▶ Übertragung auf Vogelbeobachtung



Kamerawagen-Linux

YouTube-Kanal



@kamerawagen-linux9276

Warum eigene KI-Modelle?

Das Problem mit Cloud-AI

Standard-KI-Modelle versagen bei europäischen Gartenvögeln:

- ▶ Kohlmeise → „Asiatischer Fasan“
- ▶ Rotkehlchen → „Kleine Vogelart (unbekannt)“
- ▶ Blaumeise → „Papagei“

Grund: Trainingsdaten primär für nordamerikanische/asiatische Märkte

Souveräne Edge-AI

Vollständige Kontrolle über:

- ▶ **Daten** – Lokale Speicherung, keine Cloud-Abhängigkeit
- ▶ **Modelle** – Custom-Training für regionale Vogelarten
- ▶ **Infrastruktur** – Edge-Computing auf Open-Hardware
- ▶ **Lizenzen** – MIT-lizenziert, keine Vendor-Lock-ins

Welche Modelle nutzen wir im Projekt? (v2.2.3+)

YOLOv26n (Objekterkennung)

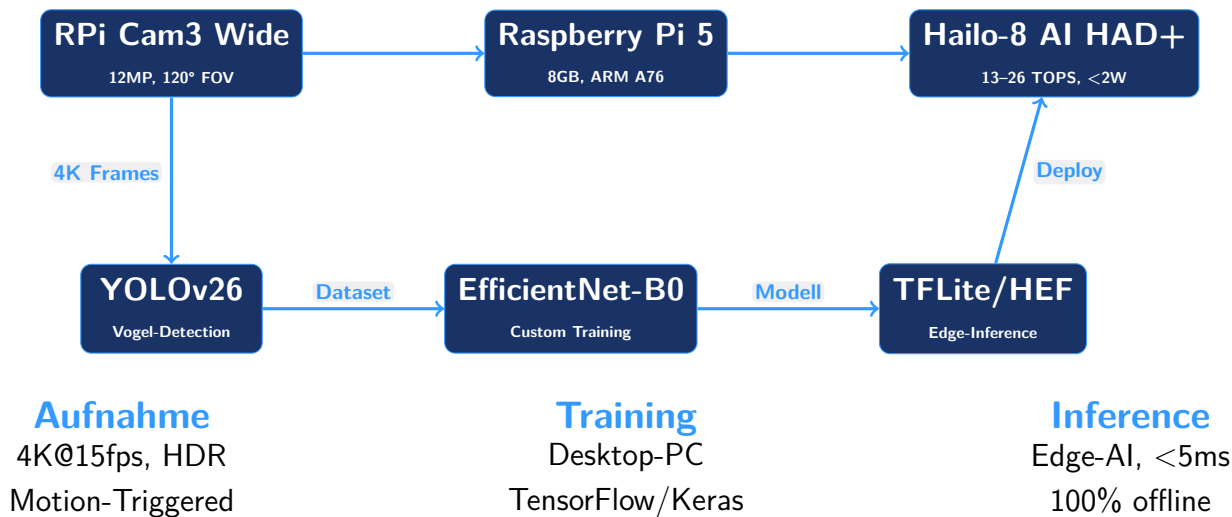
- ▶ **Aufgabe:** Objekte im Bild finden (Bounding Boxes)
- ▶ **Detection-Ziele:** Vogel, Hund, Katze, Mensch (erweiterbar auf andere Klassen)
- ▶ **Deployment:** Hailo-8 AI HAD+ (HEF-Format)
- ▶ **Performance:** <5ms pro Frame @ 30 fps

Eigenes Klassifikations-Modell

- ▶ **Aufgabe:** Vogelart bestimmen (8 Arten)
- ▶ **Architektur:** EfficientNet-B0 (Transfer Learning)
- ▶ **Training:** Mit lokalen Video-Daten
- ▶ **Accuracy:** >99% auf Test-Set

Pipeline: Kamera → YOLO-Detection (Hailo NPU) → Crop → EfficientNet → Klassifikation

System-Architektur



Hardware-Stack: Warum diese Komponenten?

Compute & AI

- ▶ **Raspberry Pi 5** (8GB)
ARM Cortex-A76, 2.4 GHz, CSI, USB3
- ▶ **Hailo-8 AI HAD+** (13–26 TOPS)
M.2 HAT+, <2W, TFLite/ONNX/PyTorch
- ▶ **Warum nicht Google Coral?**
Hailo: bessere RPi5-Integration,
M.2 nativ, aktiv gepflegt

Sensorik & Peripherie

- ▶ **Camera Module 3 Wide**
12MP, 120° FOV, Sony IMX708
- ▶ **Warum Wide-Angle?**
Futterhaus komplett erfassen,
Vögel bei Anflug erkennen
- ▶ **USB-Mikrofon + 27W PSU**
ALSA-Support, Outdoor-tauglich

Docker Deployment (v2.2.3+)

- ▶ **Container-Isolation:** Saubere Isolierung der Anwendung
- ▶ **Multi-Platform Build:** Cross-Compilation auf x86 (linux/arm64)
- ▶ **Hailo via Bind-Mount:** NPU-Libs aus dem Host, Container bleibt schlank
- ▶ **Ansible-Deployment:** Automatisiertes Image-Build + Deploy

Gesamtkosten: ~150€ | **Stromverbrauch:** ~8W | **Docker Image:** 500MB (komprimiert)

Vogelhaus-Design: Vorher vs. Aktuell

Vorhergehendes Design



Modulares 3D-Druck-Vogelhaus

Aktuelles Design



Komplett neues Design mit Fokus auf Modularität

Aktuelles Design (Adrian Imme)



Neuer 3D-Druck

- ▶ Modularer Aufbau mit 3D-Druck
- ▶ Verbesserte Kameraanordnung
- ▶ Optimierte Wetterfestigkeit
- ▶ Bessere Belüftung für RPi5
- ▶ Integrierte Kabelführung
- ▶ Einfachere Montage & Wartung
- ▶ Vogelfutter besser geschützt vor Nässe

3D-Druck-Vogelhaus-Design: Modularität

Oberteil



- ▶ Vogelfutter im separaten Behälter
- ▶ Einfaches Nachfüllen von Futter
- ▶ Bessere Wetterfestigkeit
- ▶ Klare Trennung zwischen Elektronik und Futter
- ▶ Oberteil kann unabhängig vom Unterteil zur Fütterung genutzt werden

Hardware-Modul



- ▶ Elektronik-Komponenten in einem Modul
- ▶ Einfacher Zugriff für Wartung
- ▶ Verbesserte Kabelführung
- ▶ Bessere Kühlung durch optimiertes Design
- ▶ Möglichkeit zum Austausch von Komponenten (z.B. Kamera, NPU)

Das Open-Source-Ökosystem

Drei GitHub-Repositories

vogel-kamera-linux Automatisches Kamerasystem

Raspberry Pi 5 + Hailo-8 AI HAD+, 4K-Aufnahme, Echtzeit-Klassifizierung

vogel-model-trainer Trainingsdaten-Extraktion

YOLOv26-Detection, Frame-Analyse, Dataset-Organisation

vogel-video-analyzer Qualitätskontrolle

Blur-Detection, Duplikat-Entfernung, Occlusion-Check

@kamera-linux



GitHub Repository & Version v2.2.5 (Latest)

Open Source Projekt mit Docker Container

- ▶ **Öffentliches Repository** mit vollständigem Quellcode
- ▶ **Docker Container:** Multi-Platform Build (linux/arm64)
- ▶ **Umfangreiche Docs** mit Docker + Ansible-Deployment
- ▶ **v2.2.5 Features:** 10 Parameter: Fokus + EV + AWB + 5 Bildqualitäts-Regler

vogel-kamera-linux @ GitHub

Neueste Version: v2.2.5 (21. März 2026) — Vollständige Kamera-Kontrolle



@kamera-linux/vogel-kamera-linux

Docker Web API Architektur (v2.2.5)

Container-Anforderungen

- ▶ **Basis-Image:** python:3.13-slim-bookworm
- ▶ **Flask API:** Sichere Web-GUI auf Port 8443
- ▶ **Authentifizierung:** JWT + TOTP-2FA
- ▶ **Fokus-Control:** Manueller Fokus-Slider
- ▶ **Hailo-Zugriff:** Bind-Mounts für NPU-Libs
- ▶ **Kamera-Stream:** rpicas-vid im Subprocess

Sicherheit

- ▶ HTTPS mit selbstsigniertem Zertifikat
- ▶ JWT-Token für API-Zugriff
- ▶ TOTP-basierte 2-Faktor-Auth
- ▶ Privilegierter Container für Kamera-Reset

Bind-Mounts aus dem Host

- ▶ /host/lib + /host/usr/lib – Trixie libc
- ▶ /opt/rpicam-vid – rpicas-apps Binary
- ▶ /dev/video* + /dev/media* – GPU-Zugriff
- ▶ ~/Videos – Video-Aufnahmen (Host)
- ▶ /etc/pi-daemon/certs – TLS-Zertifikat

Image-Größe:

~500MB (komprimiert)

inklusive Dependencies

Build-Pipeline auf x86 Build-Host (Gentoo/Linux)

```
# 1. QEMU + Docker Buildx Setup (einmalig)
./ansible/build_and_deploy.sh --setup-host

# 2. Multi-Platform Docker Image bauen (linux/arm64)
docker buildx build --platform linux/arm64 \
  --file docker/Dockerfile \
  --tag vogel-pi:latest --load .

# 3. Image auf Raspberry Pi bereitstellen + Container starten
./ansible/build_and_deploy.sh --install      # Erstinstallation
./ansible/build_and_deploy.sh --update      # Updates
```

Build-Host Anforderungen:

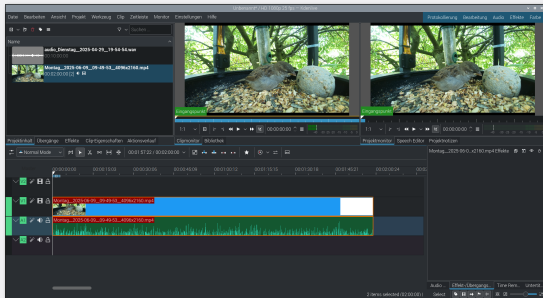
- ▶ Docker + docker-buildx
- ▶ QEMU-aarch64-Emulation
- ▶ binfmt-misc für native Compilation
- ▶ Ansible 2.10+

Build-Zeit: ~3–8 Min (neu) oder ~30 s (gecacht)

Vorteile:

- ▶ Kein Bauen auf Pi nötig (zu langsam)
- ▶ Versionskontrolle des Images
- ▶ Reproduzierbare Deployments
- ▶ Rollbacks trivial

Kdenlive Workflow



Open-Source-Videoschnitt

- ▶ Abhängig vom Video auch Audio möglich (z.B. Vogelstimmen)
- ▶ MP4 Format für YouTube
- ▶ Farbkorrektur & Stabilisierung
- ▶ Titel & Untertitel

YouTube-Kanal



@vogel-kamera-linux

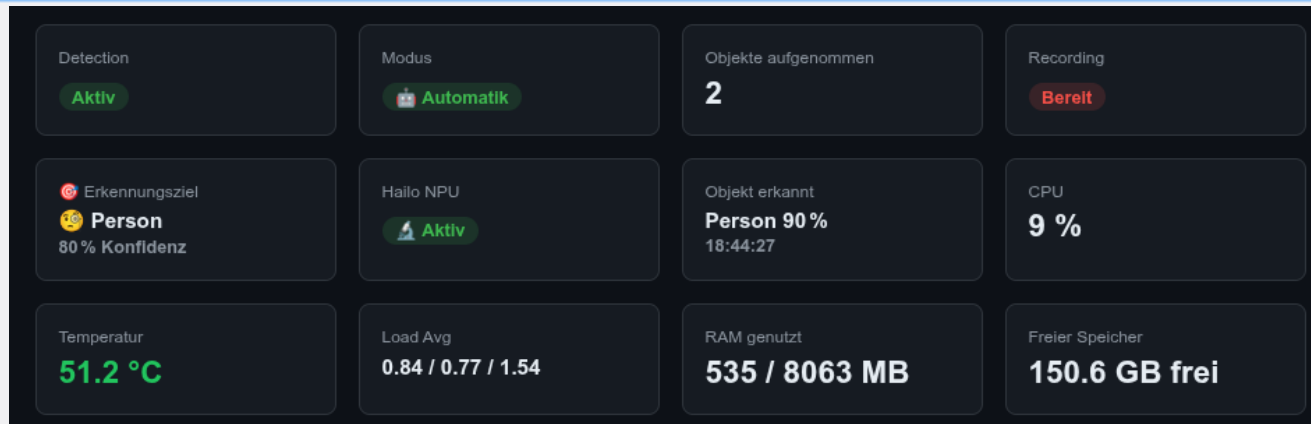
Typischer Ausschnitt aus der automatisierten Vogelbeobachtung



► Beispielvideo ab Sekunde 35

Die Wiedergabe muss ggf. manuell gestartet werden.

Hardware-Status



Werden in 5 Sekunden aktualisiert

Hardware-Status

DETECTION

▶ Detection-Modus starten ■ Detection-Modus stoppen

Erkennen: ☐ Vogel ☒ Mensch ☐ Hund ☐ Katze ☐ Alle 4

Confidence: 0.80 (0.10–0.95)

📹 Detection-Modus aktiv – bei Erkennung wird automatisch aufgenommen. Manuelle Aufnahme gesperrt. (2 Aufnahmen)

Auswahl unterschiedliche Aufnahme

Umfassende Kamera-Kontrolle (v2.2.5)

Einstellungen Aufnahmeparameter

AUFNAHME

🔒 Manuelle Aufnahme gesperrt – Detection-Modus ist aktiv.

Fokus: 5.0 (≈ 20 cm) (0.5=2m ... 10=10cm)

Belichtung (EV): 0.0 (-2=dunkel ... +2=hell)

Weißabgleich (AWB): Auto ▼

Helligkeit: 0.0 (-1=dunkel ... +1=hell)

Kontrast: 1.0 (0.5=flach ... 1=normal ... 2=knackig)

Sättigung: 1.0 (0=graustufen ... 1=normal ... 2=+)

Schärfe: 1.0 (0=keine ... 1=normal ... 2=extra)

☒ Gain (ISO): 1.0 (1=normal ... 8=max, nur bei Lowlight)

Dauer 2 min | Normal 4K (4096×2160 25fps – max) ▼ Aufnahme Audio

Konvertieren Übertragen

Werden in Echtzeit angewendet

Mehrere Zugriffsmöglichkeiten

Einstellungen SSH Verbindung

Ziel 5

☒ Aktiv

Host / IP

SSH Benutzer

192.168.1.100

roimme

Ziel-Pfad (Remote)

/home/roimme/Videos/

SSH Private Key (leer lassen = Key-Auth deaktiviert)

-----BEGIN OPENSSH PRIVATE KEY-----
...
-----END OPENSSH PRIVATE KEY-----

 Testen

+ Ziel hinzufügen

 Alle speichern

Übertragung der Video und Audio-Dateien auf den Host

Problem

Wie entsteht aus Rohdaten ein trainiertes ML-Modell?

Lösung: Drei Kernfunktionen

- 1. Extract:** Vögel aus 4K-Videos extrahieren
YOLOv26-Detection, Qualitätsfilter
- 2. Organize:** Train/Validation Dataset erstellen
Stratifizierte Splits (z. B. 80/20)
- 3. Train:** EfficientNet-B0 trainieren
Transfer Learning, Data Augmentation

Repository



@kamera-linux/vogel-model-trainer

PyPI: vogel-model-trainer

CLI-Workflow: Automatisierte Dataset-Erstellung

```
# Extraktion + Hintergrundentfernung + Duplikat-Erkennung
vogel-trainer extract ~/Videos/garten.mp4 \
  --folder ~/training-data/ \
  --bird kohlmeise \
  --threshold 0.5 \
  --sample-rate 3 \
  --remove-background \
  --min-sharpness 150 \
  --min-edge-quality 80 \
  --deduplicate
```

Extraktions-Features:

- ▶ YOLOv26 Vogel-Detection
- ▶ Bounding-Box Cropping (20% Padding)
- ▶ KI-Hintergrundentfernung (rembg)
- ▶ Perceptual Hashing (Duplikate)

Qualitätsfilter:

- ▶ Unschärfe-Erkennung (Laplacian)
- ▶ Min. Schärfe-Score (Threshold)
- ▶ Kanten-Qualität (Sobel-Gradient)
- ▶ Box-Größe (Min/Max Pixel)

Ergebnis: Sauberes Dataset ohne manuelle Sichtung

Was das Tool leistet

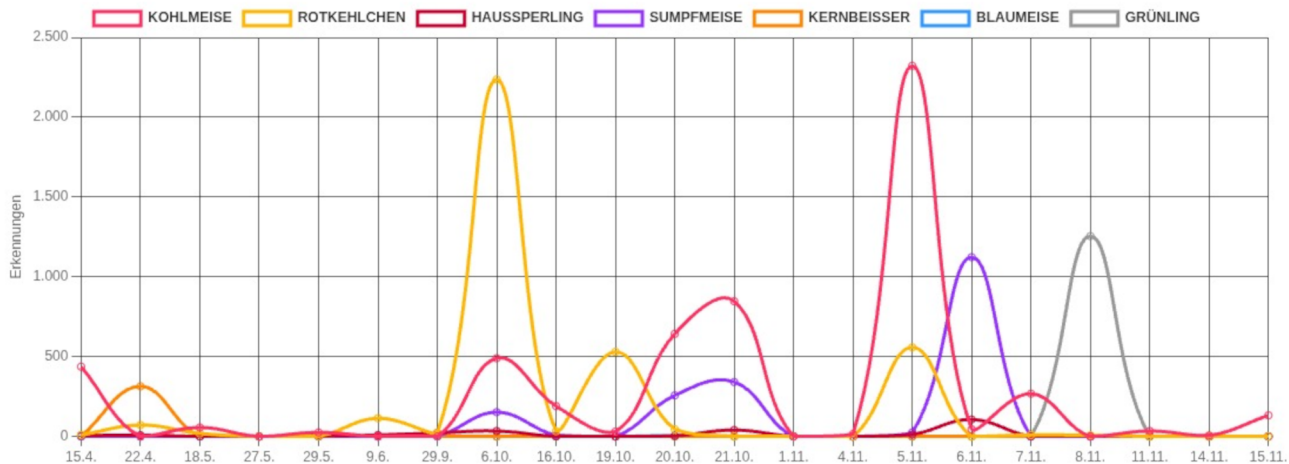
- ▶ YOLOv26-basierte Erkennung von Vogelinhalt in Videos
- ▶ Optional: Artenklassifikation via Hugging Face Modelle
- ▶ HTML-Report mit Timeline, Artenverteilung und Thumbnails
- ▶ JSON-Export für Automatisierung und Archivierung
- ▶ Optional annotierte Videos (Bounding Boxes + Labels)

Ziel: Batch-Analyse großer Videomengen mit reproduzierbaren Ergebnissen



Entwicklung einzelner Arten

Wie oft wurde welche Art über die Zeit erkannt?



Typischer CLI-Workflow

```
# Analyse + Artenklassifikation + HTML-Report
vogel-analyze --language de \
  --identify-species \
  --species-model kamera-linux/german-bird-classifier-v2 \
  --species-threshold 0.80 \
  --html-report report.html \
  --sample-rate 15 \
  --max-thumbnails 12 \
  video.mp4
```

- ▶ Schnellere Verarbeitung über `--sample-rate`
- ▶ Optional annotiertes Video über `--annotate-video`
- ▶ Batch-Modus für ganze Verzeichnisse (z. B. `**/*.mp4`)

Warum das Repo für unser Projekt wichtig ist

Beitrag in unserer Gesamtpipeline

- ▶ Vorfilterung: Videos ohne Vogelinhalt zuverlässig aussortieren
- ▶ Qualitätskontrolle vor dem Model-Training
- ▶ Einheitliche Reports für Vergleich über Zeitreihen
- ▶ Saubere Datengrundlage für `vogel-model-trainer`

Ergebnis: weniger manuelle Sichtung, schnellere Iteration, bessere Datensätze

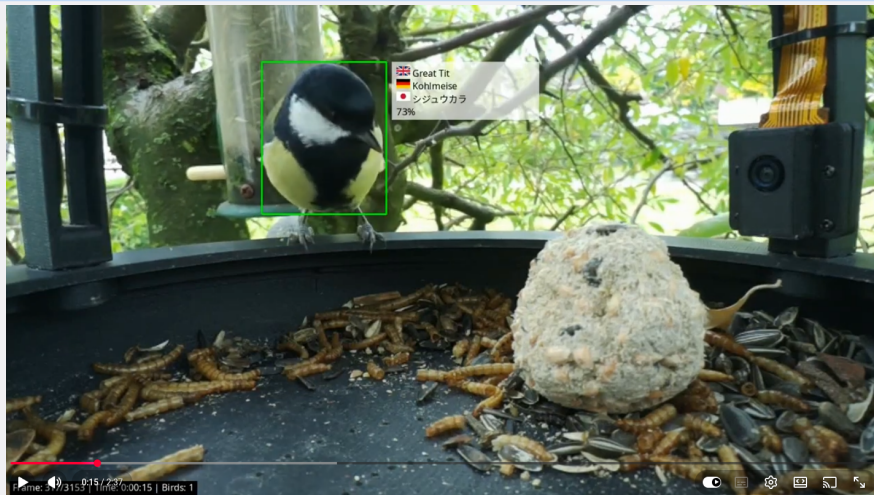
Repository



@kamera-linux/vogel-video-analyzer

PyPI: `vogel-video-analyzer`

Demo des Analyzer-Workflows auf YouTube



► Beispielvideo ab Sekunde 15

Die Wiedergabe muss ggf. manuell gestartet werden.

Was wir erreicht haben

- ▶ **End-to-End ML-Pipeline:** Von der 4K-Aufnahme bis zum Edge-Inference
- ▶ **Hailo-8 AI HAD+:** 4.8 ms Inference, 30+ fps Durchsatz, <2W Power
- ▶ **Docker Web API:** Sichere HTTPS-GUI mit JWT+TOTP-2FA auf Port 8443
- ▶ **10-Parameter Kamera-Kontrolle:** Fokus + EV + AWB + Brightness + Contrast + Saturation + Sharpness + Gain (REMOTE-getestet!)
- ▶ **Automatisiertes Deployment:** Ansible-basiertes Build & Deploy auf Raspberry Pi
- ▶ **Multi-Objekt-Erkennung:** Vogel, Hund, Katze, Mensch (erweiterbar)

Kernbotschaften: Souveräne Edge-AI in der Praxis

Das mitnehmen

- ▶ **Containerisierung** macht Edge-AI reproducierbar und wartbar
- ▶ **Souveräne Edge-AI** ist praktisch umsetzbar – auch Industrial-Grade
- ▶ **Kontinuierliche Verbesserung:** v2.2.5 mit umfassender Bildqualität-Kontrolle und Feedback-Loop
- ▶ **Multi-Platform Builds** (QEMU + Docker Buildx) für ARM64-Deployment
- ▶ $\sim \$150$ *Hardware* + *DockerschlgtCloud* – *APIs*
- ✓ **Vollständige Kontrolle** über Daten, Modelle und Lizenzen

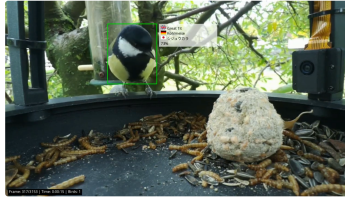


Fragen?

Wir klären diese direkt am Stand!

Diskussionen, Ideen & Erfahrungsaustausch willkommen

Vielen Dank!



Fragen, Diskussion & Networking

GitHub Repository:



@kamera-linux/vogel-kamera-linux

YouTube-Kanal:



@vogel-kamera-linux

Adrian Imme & Roland Imme